16. Open Spring Boot Project in IntelliJ

In the previous video, we downloaded a zip file from Start.spring.io with a few different libraries selected. And here what we want to do is come into IntelliJ. And in IntelliJ we want to open up that project. So we just want to come to file click on open. I can come into my spring six directory. And here I've copied all the files into my folder of spring six web app. You can see here these are the files that were in the zip file. And in IntelliJ I just have to say open. And it's going to open this up as a project. It will detect the Maven Pom in there and load up that project. So now I'm just going to click on open. And had a little error there. We can see that Maven wrapper was installed. So that is all good. We can close that. We can close out the tips as well and close that dialog and that down. And here because I do have I have this as a GitHub repository that these files have not been checked in. So that's why they're red. I will be checking these into GitHub for your reference, and there will be a link to the repository in the course resources. But here what I want to do is take a look at something first. You can see the structure here the directory structure. So how things are laid out. Let me toggle over to Chrome. This is a standard directory layout for Apache Maven. That's what we're going to be following. Maven files it. And also by default Gradle will file this. And this is really how things are organized by default. So Spring Boot is following a default structure that was established by the Maven project. I was working in Java before this existed, and every project was a little bit different. So this is a really nice layout because your build tools, this gives your build tools a standard. So the standard is source main Java. That's where your Java source files are going to be. Source main resources. This is going to be any configuration files, any text files, any property files, things like that. And then we don't see filters used very much. These are resource filter files. I honestly never used that web app. That is going to be if you are building a war file. We do not need that in a spring boot context because we'll be running with embedded Tomcat. And then also we have source test. So similar structure source test Java source test resources and filters. These are going to be for testing. We will definitely be getting that into that in the course and then source IT integration test. We won't see that as well as assembly and site. These are different Maven options. You can have Maven generate a website for your project. We won't be looking at that in the course.

And then also we have project license and then a readme dot txt. Here we will be using a readme MD for display on GitHub. But the primary thing that we are going to be concerned with is source main Java. Those are the main application files. Source test Java. Those are going to be the test classes that we write. So let's toggle back over to IntelliJ. And here brought up IntelliJ. We want to take a look at the Pom file that was generated. We briefly looked at this on the website. You can see here we are inheriting dependencies from the spring Boot starter parent. So this is going to give us what's called curated dependencies from Spring Boot. So that's going to be brought into the project. You can see here we are sending to Java 17. We selected Spring Data JPA. So we are getting a spring boot starter for JPA as well as a spring boot starter for web and then the H2 in-memory database. So now what I want to do is come over to Maven. And here I'm going to come in and take a look at the dependencies. First we want to take a look at the spring boot starter for web. So this is a spring boot starter. This gives us a set of curated dependencies. See we come in with spring boot starter. That gives us spring boot Spring boot auto configuration will be exploring that more in the course. A logging default. We have the cart annotation API, and here you can see that we are bringing Spring Core 6.0. So that's where we get Spring Framework six. The starter for web also brings in JSON. This is going to bring in our Jackson dependencies. We also get a embedded Tomcat instance. And let me collapse these so we can see a little bit more. We have the spring web. So this brings in spring beans spring core as well as spring web MVC. So this brings in Spring Web MVC and all the related dependencies. What's important about this is before Spring Boot you were responsible for figuring out all these dependencies here. We don't need to do this. We bring in the starters. The starters bring in a set of curated dependencies by the spring team. So not only do we get the proper, necessary dependencies being brought into our project, but we also get the proper versions and more importantly, and let me collapse this. The final plugin that we want to look at is our final dependency is the Spring Boot Starter test. Here we can see that this is bringing in JUnit five Mockito JSON assert spring test. So we not only get JUnit and also Mockito. We'll be looking at these coming up in the course as well, but we do get everything that we need for doing test assertions and some auto configuration for testing. So we get a nice library there. We have a lot of tools by default being brought into our application that will be available there for our use. Then of course we are getting the H2 in-memory database. This is your Java main class. So I'll go ahead and collapse that. The other things that the spring boot starter will include is a web

application. So this is what will start up the spring boot application. So this is your starting point for the application. And then you do get a JUnit test by default. And don't understand this yet. this, but this spring goes through and when it starts up, it builds what's called a context. And this is going to get initialized. And this test just makes sure that the spring configuration is happy as far as being able to start up. Now there's a lot of things that can happen if you have something misconfigured. Startup can fail. So this is really just kind of a sanity test to make sure that everything is able to start up. And then finally we do get a help. This is also brought down and the help can be handy. You can see here we're using Apache Maven. There's a link to Maven plugin reference guide OCI spring web spring data JPA. So you do have a number of basically links to getting started tutorials. And if you wish you could go ahead and explore that. There's some pretty good information there that you might want to read on your own with Spring Framework. So this right now I have not touched anything. I only downloaded that zip file from Springtime with the selected dependencies. I am going to check that into the GitHub repository. I do want to point out one of the functions of git. So inside the project structure inside of GitHub you will see a resources folder. But by default the way GitHub behaves it does not bring in empty directory. So when I load this up into GitHub, static and templates will not be there. If you do need to add those resources from something that you checked out in GitHub, you will need to manually create these two directories. These are default directories that Spring Boot will be looking for. If we have static web resources or if we have time leaf templates. So those are two things that we'll need to look at. Right now we don't need to be concerned with that. We will create those if we need it coming up in the course. But do want to point out that this is as is that I downloaded from Start.spring.io. So this is the default project that we get created for us from the spring application.

> O instrutor, usa o Sistema Operacional macOS. Eu, Juliane, como aluna, estou usando o Sistema Operacional Windows 11.

```
spring-6-webapp/
 ├── .mvn
 ├── src/
 |    ├── main/
 |    |    ├── java/
 |    |    |    └── guru.springframework.spring6webapp/
```

```
│    │    │        └── Spring6WebappApplication.java
│    │    └── resources/
│    │         ├── static/
│    │         ├── templates/
│    │         └── application.yaml
│    └── test/
│         ├── java/
│         │    └── guru.springframework.spring6webapp/
│         │        └── Spring6WebappApplicationTests.java
│         └── resources/
├── .gitattributes
├── .gitignore
├── HELP.md
├── mvnw
├── mvnw.cmd
├── pom.xml
└── README.md
```

> Inicialmente, o projeto estava com o nome `spring-6-webapp`, mas como foi lançado a versão 7 do Spring Framework, o nome do projeto passou para `spring-7-webapp`.
> A versão do Java que é apresentada nesta aula é a 17, mas com o lançamento da 25, o código foi atualizado para a versão mais atual do Java.

Abra um projeto Spring Boot no IntelliJ.

No vídeo anterior, fizemos o download de um arquivo ZIP a partir do site start.spring.io, com algumas bibliotecas selecionadas. Agora, o que queremos fazer é abrir esse projeto no **IntelliJ IDEA**.

No IntelliJ, basta acessar o menu **File** e clicar em **Open**. Em seguida, posso navegar até o meu diretório *spring six*. Aqui, eu copiei todos os arquivos para a pasta *spring six web app*. Você pode ver que esses são exatamente os arquivos que estavam dentro do arquivo ZIP. No IntelliJ, basta clicar em **Open**, e o projeto será aberto.

O IntelliJ irá detectar automaticamente o **POM do Maven** presente no projeto e carregá-lo. Agora, vou simplesmente clicar em **Open**. Ocorreu um pequeno aviso, mas podemos ver que o **Maven Wrapper** foi instalado corretamente,

então está tudo certo. Podemos fechar essa mensagem. Também podemos fechar as dicas iniciais e encerrar essas janelas de diálogo.

Neste caso, como este projeto está associado a um repositório no **GitHub** e esses arquivos ainda não foram versionados (*checked in*), eles aparecem em vermelho. Eu farei o *commit* desses arquivos no GitHub para sua referência, e haverá um link para o repositório nos recursos do curso.

Antes disso, quero analisar um ponto importante: a **estrutura de diretórios** do projeto. Aqui você pode ver como os diretórios estão organizados. Vou alternar para o navegador Chrome. Esta é a estrutura de diretórios padrão do **Apache Maven**. É exatamente esse padrão que vamos seguir. O Maven utiliza esse layout e, por padrão, o Gradle também adota uma estrutura muito semelhante. Essa é a forma como os projetos são organizados por padrão.

O Spring Boot segue uma estrutura padrão que foi estabelecida pelo projeto Maven. Antes disso, quando eu trabalhava com Java, cada projeto possuía uma organização diferente. Por isso, esse layout é muito útil, pois fornece um padrão para as ferramentas de build.

O padrão define que:

- **src/main/java** é onde ficam os arquivos-fonte Java da aplicação;
- **src/main/resources** contém arquivos de configuração, arquivos de texto, arquivos de propriedades e recursos em geral;
- o uso de **filters** não é muito comum; esses são arquivos de filtro de recursos, e, sinceramente, eu nunca utilizei isso em aplicações web;
- o diretório **webapp** é usado quando estamos construindo um arquivo **WAR**, o que não é necessário no contexto do Spring Boot, pois utilizamos um **Tomcat embarcado**.

Também temos o diretório **src/test**, com uma estrutura semelhante:

- **src/test/java** para as classes de teste;
- **src/test/resources** para recursos de teste;
- filtros de teste, quando aplicável.

Durante o curso, vamos trabalhar bastante com testes. Também existem diretórios como **src/it** (testes de integração), **assembly** e **site**, que são opções específicas do Maven. O Maven, por exemplo, permite gerar um site para o projeto, mas isso não será abordado neste curso.

Além disso, temos arquivos como a licença do projeto e o **README**. Neste curso, utilizaremos um **README.md** para exibição no GitHub.

O principal foco será:

- **src/main/java**: arquivos principais da aplicação;
- **src/test/java**: classes de teste que iremos escrever.

Agora, voltando ao IntelliJ, vamos analisar o arquivo **pom.xml** que foi gerado. Já vimos esse arquivo rapidamente no site. Aqui podemos observar que o projeto herda dependências do **spring-boot-starter-parent**, o que nos fornece o que chamamos de **dependências curadas** pelo Spring Boot. Essas dependências são automaticamente incluídas no projeto.

Podemos ver que o projeto está configurado para utilizar **Java 25**. Selecionamos o **Spring Data JPA**, portanto temos o *starter* correspondente, além do *starter* para **Spring Web** e o banco de dados **H2 em memória**.

Agora, vou acessar a aba do Maven para analisar as dependências. Primeiro, vamos observar o **Spring Boot Starter Web**. Esse *starter* fornece um conjunto de dependências curadas. Ele inclui:

- o próprio Spring Boot;
- a **auto-configuração do Spring Boot**, que será explorada mais adiante no curso;
- configuração padrão de logging;
- a API de anotações Jakarta;
- o **Spring Core**, que faz parte do **Spring Framework 7**.

O *starter* web também inclui suporte a **JSON**, trazendo as dependências do **Jackson**, além de uma instância embarcada do **Tomcat**.

Ao expandir as dependências, vemos também o **Spring Web**, que inclui **Spring Beans**, **Spring Core** e **Spring Web MVC**, juntamente com todas as dependências relacionadas.

O ponto mais importante aqui é que, antes do Spring Boot, o desenvolvedor precisava definir manualmente todas essas dependências. Com o Spring Boot, basta utilizar os *starters*, que trazem um conjunto de dependências curadas pela equipe do Spring, com versões corretas e compatíveis entre si.

Outra dependência importante é o **Spring Boot Starter Test**. Ele inclui:

- **JUnit 5**,
- **Mockito**,
- **JSON Assert**,
- **Spring Test**.

Essas bibliotecas fornecem tudo o que precisamos para escrever testes e realizar asserções, além de oferecer auto-configuração específica para testes. É um conjunto muito completo de ferramentas disponíveis por padrão.

Além disso, temos o banco de dados **H2 em memória**.

A seguir, temos a classe principal Java da aplicação. Ela é responsável por inicializar a aplicação Spring Boot, sendo o ponto de entrada do sistema.

Por padrão, também é gerado um teste JUnit. Mesmo que você ainda não compreenda totalmente esse teste, ele verifica se o **contexto do Spring** é carregado corretamente. Durante a inicialização, o Spring cria o que chamamos de *Application Context*. Se houver alguma configuração incorreta, a inicialização pode falhar. Esse teste funciona como um teste de sanidade, garantindo que a aplicação consiga iniciar corretamente.

Por fim, temos o arquivo **HELP.md**, que também é incluído por padrão. Ele contém informações úteis, como:

- indicação de que o projeto utiliza **Apache Maven**;
- links para o guia de referência do plugin Maven;
- tutoriais introdutórios sobre Spring Web e Spring Data JPA.

Esses links podem ser explorados por conta própria, pois contêm informações bastante relevantes sobre o Spring Framework.

Até este momento, eu não alterei absolutamente nada no projeto. Apenas fiz o download do arquivo ZIP a partir do **Start.spring.io**, com as dependências selecionadas. Vou versionar esse projeto no GitHub.

Quero destacar um comportamento importante do **Git**: dentro da estrutura do projeto existe a pasta **resources**, mas, por padrão, o Git não versiona diretórios vazios. Portanto, ao subir o projeto para o GitHub, os diretórios **static** e **templates** não aparecerão. Caso você precise desses diretórios em um projeto clonado do GitHub, será necessário criá-los manualmente.

Esses diretórios são utilizados pelo Spring Boot para:

- recursos estáticos (como arquivos CSS, JavaScript e imagens);
- templates, como os utilizados pelo **Thymeleaf**.

Neste momento, não precisamos nos preocupar com isso. Caso seja necessário ao longo do curso, esses diretórios serão criados.

Portanto, este projeto está exatamente como foi gerado pelo **Start.spring.io**. Esse é o projeto padrão que o Spring Boot cria automaticamente para nós.

Se quiser, posso:

- **comparar o layout Maven com projetos Gradle no Spring Boot 4**,
- **explicar o papel de cada starter em nível arquitetural**,
- ou **mapear diferenças práticas entre IntelliJ no macOS e no Windows 11** para acompanhar o curso sem atritos.